

AIKernel Phase-2 Theory: Semantic Compilation Architecture

意味状態・観測可能性・統治された遷移のための Semantic Runtime Theory

Takuya Sogawa

2026-06-06

AIKernel Phase-2 Theory: Semantic Compilation Architecture

意味状態・観測可能性・統治された遷移のための Semantic Runtime Theory

Author: Takuya Sogawa

ORCID: <https://orcid.org/0009-0009-7499-2595>

Version: v0.2.0

Publication date: 2026-06-06

DOI: <https://doi.org/10.5281/zenodo.20312092>

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Canonical language: English

Companion translation: Japanese

Series position: AIKernel / AIOS Phase-2 Theory

Target: AIKernel / AIOS Semantic Runtime

Proposed namespace: AIKernel.Abstractions.Semantics

Stability: Experimental / Non-normative

1. Abstract

本稿は、AIKernel / AIOS Phase-2 における **Semantic Compilation Architecture** を定義する。Phase-1 は、知識同一性、セマンティックストレージ、推論前受理統治、および軌道ガバナンスからなる AIOS の静的アーキテクチャを定義した。これに対し Phase-2 は、それらの境界を一般化し、AIOS が実行時および合成時に「意味」をどのように扱うかを定義する Semantic Runtime Theory を提示する。

本稿の中心命題は、AIOS が自然言語の人間意図をそのまま実行しない、という点にある。AIOS は、人間の意図を観測可能な意味状態へ変換し、統治制約の下で意味遷移を評価し、許容された意味遷移のみを実行へ接続する。本モデルにおいて、LLM / SLM は実行権限を持つ主体ではなく、意味状態、意味遷移、または回路候補の提案者として扱われる。AIKernel は、それらの提案を評価する決定論的な統治境界である。

本稿は、Semantic State、Semantic IR、Governed Circuit、Prototype Space、Semantic Compression、Runtime Policy、および Deterministic Synthesis を第一級のアーキテクチャ概念として導入する。さらに、Paper 04 で定義した Semantic Ellipsoid を、複合距離関数における意味状態成分として接続し、Semantic Compilation における Quantization の数理と整合させる。

本稿は、人間の意味理解を完全に形式化するものではない。また、Embedding 空間が人間の意味を完全に表現するとも、Semantic Compilation が事実正確性を証明するとも主張しない。本稿が定義するのは、高エントロピーな人間意図を、有限で、観測可能で、統治可能で、再現可能なランタイム構造へ変換するための、有界で運用可能な理論である。

2. AIKernel フェーズ構造における位置づけ

AIKernel は、次の三つの理論フェーズとして構成される。

Phase-1 = Static Architecture / OS Specification

Phase-2 = Semantic Runtime Theory / Semantic Compilation Architecture

Phase-3 = Foundation / Trust, Proof, Cryptographic Layer

Phase-1 は、AIOS の構造を定義する。Phase-2 は、AIOS が意味をどう扱うかを定義する。Phase-3 は、AIOS を社会的・暗号的にどう信頼可能にするかを定義する。

本稿は、Phase-1 の静的アーキテクチャ論文群と、Phase-3 の Foundation 技術ノート群の間に位置づけられる。目的は、AIOS の統治境界を、観測可能な意味状態遷移および将来的な信頼指向拡張へ接続するための Semantic Runtime Theory を定義することである。

2.1 Phase-1 との関係

Phase-1 は、以下の四つの静的境界を定義する。

Phase-1 paper	Boundary	Phase-2 から見た役割
Paper 01: ROM Format and Knowledge Snapshot Model	Knowledge identity	Semantic State が参照する不変知識単位
Paper 02: VFS Architecture and Semantic Storage Model	Storage boundary	意味状態アーティファクトの保存・取得境界
Paper 03: Pre-Inference Admissibility Governance	Entry boundary	意味遷移開始前の決定論的受理判定
Paper 04: Trajectory Governance Model	Runtime governance	意味軌跡と候補行動の運用上の評価

Phase-2 は、これらの境界を Semantic Runtime として一般化する。特に、Semantic IR の4スロットは、Phase-1 の4つの境界に対応するように設計される。

Semantic IR = (G, T, C, B)

G = Graph topology -> Paper 01 / ROM dependency topology

T = Capability types -> Paper 02 / VFS and provider capabilities

C = Governance constraints -> Paper 03 / admissibility and policy constraints

B = Boundary invariants -> Paper 04 / trajectory and runtime safety invariants

これは単なる整理ではない。Phase-1 は、Phase-2 が意味を統治されたランタイム構造へコンパイルするために必要な静的な器を提供している。

2.2 Paper 04 との関係

Paper 04 は、スコア、閾値、リスク信号、および ReplayLog によって推論軌跡を評価するための運用上のガバナンスモデルを定義した。本稿は、その運用モデルを一般化し、意味状態、観測可能性、および統治された遷移を第一級のアーキテクチャ概念として扱う Semantic Runtime Theory へ昇格させる。

Paper 04:

Trajectory Governance のための運用モデル

- スコア

- 閾値

- リスク評価

- Abort / Permit / Clarify

- ReplayLog

Phase-2:

Semantic Runtime Theory のための一般理論

- Semantic State

- Semantic Transition

- Observability Primitive

- Runtime Policy

- Semantic Compilation

Paper 04 は governance algorithm であり、Phase-2 はそのアルゴリズムが動作する semantic runtime theory である。

2.3 Phase-3 との関係

Phase-3 は Phase-2 理論の成立に必須ではない。Phase-3 は、同一性、来歴証明、暗号的アドレス、Digital Deed、および信頼指向の AIOS 拡張を扱う将来の Foundation 層として位置づけられる。

Phase-2 で定義される Semantic State、Semantic IR、Governed Circuit、Runtime Policy、および ReplayLog は、将来的に Phase-3 の ROM Identity、Digital Deed、PQC Address、Anti-Slop Architecture、および provenance verification へ接続され得る。ただし、本稿はそれらを完全に定義するものではない。

3. 問題設定: Prompt Execution から Semantic Runtime へ

従来の LLM アプリケーションやエージェントフレームワークでは、自然言語プロンプトが実行計画、ツール選択、コード生成、または外部システム操作へ直接接続されることが多い。この構造では、高エントロピーな自然言語要求が確率的生成器へ渡され、その出力が後段のシステム境界を駆動してしまう。

この設計には、少なくとも以下の構造的問題がある。

1. **意味境界の欠如:** システムは、説明、計画、実行、破壊的意図、仮説的思考を十分に区別できない。
2. **状態遷移の非観測性:** システムは、どの意味状態からどの意味状態へ移ろうとしているかを明示的に観測できない。
3. **プロンプトと仕様の混同:** プロンプトは高エントロピーな意図表現であるにもかかわらず、実行仕様のように扱われる。
4. **合成過程の非決定性:** ツールグラフ、ワークフロー、コード構造、エージェントループが確率的に構成される。
5. **再現可能な意味変換の欠如:** 人間の意図がなぜ特定の実行トポロジーへ写像されたのかを後から再構成できない。

AIKernel Phase-2 は、この問題を Prompt Execution から Semantic Runtime への転換として扱う。AIOS は自然言語を直接実行するのではなく、人間の意図を観測可能で、統治可能で、再現可能な意味遷移へコンパイルする。

この中心命題は、次のように要約できる。

Observability -> Governability -> Replayability

観測できない意味状態は統治できない。統治できない遷移は、監査可能な証跡として再現できない。したがって Phase-2 は、まず何を観測可能にするかを定義する。

4. Scope and Non-Claims

4.1 Scope

本稿は、以下を定義する。

- Semantic State
- Semantic IR
- Semantic Transition
- Observability Primitive
- Governed Circuit と Prototype Space
- Semantic Compression / Semantic Quantization
- Runtime Policy と Admissibility
- 固定写像における Deterministic Synthesis
- Phase-1 静的アーキテクチャとの接続
- Phase-3 Foundation への橋渡し

4.2 Non-Claims

本稿は以下を主張しない。

- 人間の意味理解を完全に形式化できる
- Semantic Ellipsoid が人間の意味を完全に表す
- Semantic IR があらゆる自然言語意図を損失なく表現する
- 意味遷移の大域的な数学的安定性を証明する
- LLM 内部の推論過程を完全に決定論化する
- Phase-3 の暗号、PQC、Digital Deed、Provenance を本稿で解決する
- 量子計算が Semantic Compilation の正しさに必須である

本稿の主張はより限定的である。AIKernel は、統治境界上の意味を、観測可能で、制約可能で、再現可能な、有界のランタイムアーティファクトへ変換できる。

5. 意味的エントロピーの運用上の近似

自然言語で表現された人間意図は、多数の解釈、ツール選択、データ依存関係、実行経路を含み得る。入力意図 u に対して、到達可能な許容意味実行軌跡の集合を次のように表す。

$Traj_adm(u) = \{ \text{admissible semantic execution trajectories reachable from } u \}$

ここで意味的エントロピー $H_{sem}(u)$ は、人間の意味そのものに対する厳密な Shannon entropy ではない。これは AIKernel の統治境界上で利用可能な許容軌跡の多様性を表す operational proxy である。

$H_{sem}(u) \sim \log |Traj_adm(u)|$

Semantic Compilation の目的は、人間の意味を哲学的に一意へ還元することではない。実行トポロジーに関する運用上の不確実性を、明示的な統治制約の下で減衰させることである。

5.1 高エントロピーな意図

未構造化プロンプトは、多数の解釈へ展開し得る。

User intent

- > multiple interpretations
- > multiple tools
- > multiple context scopes
- > multiple execution paths

この状態で LLM の自律ループに直接委ねると、軌跡漂流、権限逸脱、無制限のツール呼び出し、またはワークフロー構造の不安定化が発生し得る。

5.2 有界な Semantic Runtime

AIKernel は、意味的エントロピーが哲学的な意味でゼロになるとは主張しない。主張するのは、実行に関係する不確実性をランタイム境界上で有界化できる、という点である。自然言語意図は Semantic IR、Prototype Space、Governed Circuit、Runtime Policy へ段階的に写像され、監査可能な有限の遷移候補へ変換される。

6. Semantic State Model

Semantic Runtime には、意味の観測可能な表現が必要である。AIKernel は、観測可能な意味状態を実装上の近似として次のように表す。

$s_t = (\mu_t, \Sigma_t)$

ここで、 μ_t は時点 t における代表意味ベクトル、 Σ_t は不確実性の推定値であり、実装上は対角共分散行列として近似されることが多い。

このモデルは、Paper 04 で定義した注意深い前提を継承する。Embedding 空間は、人間の意味を完全に表すものとは仮定されない。Semantic State は、意味の形而上学的定義ではなく、観測可能性のためのプリミティブである。すなわち、意味のばらつきを AIOS の統治境界で観測し、制約し、比較し、リプレイ可能にするために導入される近似モデルである。

6.1 Semantic Ellipsoid

Semantic Ellipsoid は、意味状態の周囲に定義される許容領域である。これは完全な意味ではなく、観測された意味的ばらつきを表す。

$$E_t(k) = \{x \in V \mid d(x, s_t) \leq k\}$$

実装上、距離成分は対角分散と安定化定数を用いて近似できる。

$$d(x, s_t) = \sqrt{\sum_j ((x_j - \mu_{t,j})^2 / (\sigma_{t,j}^2 + \epsilon))}$$

この定式化により、AIKernel は遷移が統治された意味領域内に留まっているかを観測するための運用上の方法を得る。

6.2 観測可能性からコンパイル可能性へ

Paper 04 は Semantic Ellipsoid を用いて推論軌跡を観測・統治した。Phase-2 は、この観測可能性の原理をより上位で再利用する。すなわち、未検証の Semantic IR と候補 Governed Circuit との適合性評価において、同じ意味状態の不確実性が利用される。

Observability becomes compilability.

これが、Phase-1 のランタイムガバナンスと Phase-2 の Semantic Compilation を接続する中心命題である。

7. Semantic IR

高エントロピーな概念仕様は、統治可能になる前に、決定論的な中間表現へ写像されなければならない。本稿では、**Semantic IR** を4スロットのタプルとして定義する。

$$X = G \times T \times C \times B$$

$$x = (G, T, C, B)$$

4つのスロットは以下である。

Slot	Name	Meaning	Phase-1 correspondence
G	Graph Topology	データフロー、推論依存、実行トポロジー	Paper 01
T	Capability Types	必要な provider、tool、VFS access、solver capability	Paper 02
C	Governance Constraints	ポリシー制限、リスク閾値、予算、受理規則	Paper 03
B	Boundary Invariants	Fail-closed 条件、事前条件、事後条件、runtime invariant	Paper 04

Semantic IR は自然言語の要約ではない。比較、制約、合成、リプレイが可能な、統治された構造表現である。

7.1 なぜ4スロットなのか

この4スロット構造は、本来分離すべき関心をプロンプト内に混在させないために設計されている。

- G は実行トポロジーを記述する。
- T は必要な能力を記述する。

- C は実行を制約するポリシーを記述する。
- B は決して破ってはならない不変条件を記述する。

この分離により、受理判定、ガバナンス、合成、リプレイが明示化される。

7.2 関連する意味状態

Semantic IR は離散的な構造アーティファクトである。観測可能な意味状態は、この IR と関連づけられるが、第5スロットとして IR に混入させない。

Semantic IR: $x = (G, T, C, B)$

Associated semantic state: $s = (\mu, \Sigma)$

この区別は重要である。構造的 IR はトポロジー、能力、制約、不変条件を保持する。一方で、Semantic State はその IR に関連する意味の観測可能な不確実性を表す。

Concept specification / human intent



Semantic IR: $x = (G, T, C, B)$ [離散的な構造アーティファクト]

| associated with



Semantic state: $s = (\mu, \Sigma)$ [観測可能な意味的不確実性]

| compared against



Governed circuit: $c_k = (G_k, T_k, C_k, B_k)$ with s_k

この図はデータフロー図ではない。Semantic IR と Semantic State が、関連しているが同一ではないアーティファクトであることを示すための概念図である。前者は構造的なコンパイル対象であり、後者は適合度と不確実性を評価するための観測モデルである。

8. Governed Circuits and Prototype Space

Governed Circuit とは、形式的に許容され、有界であり、Fail-Closed が保証された意味論的実行トポロジーである。

無制約なエージェントループとは異なり、Governed Circuit は、能力、ポリシー、境界不変条件が明示された検証済みの実行構造である。例として、Retrieval Circuit、Quarantine Circuit、Cross-Verification Circuit、Deterministic Solver Circuit、Read-Only Summarization Circuit などがある。

Prototype Space P は、このような検証済み回路の有限集合である。

$P = \{c_1, c_2, \dots, c_N\} \subset X$

各候補回路は、同じ4スロットの Semantic IR 空間で表現される。

$c_k = (G_k, T_k, C_k, B_k)$ in X

この性質により、未検証の入力 IR x と、検証済みの回路候補 c_k を同じ構造空間上で比較できる。

8.1 閉じているが拡張可能な空間

安全性のため、Prototype Space は単一のコンパイル判断中には閉じている。コンパイラは、その時点で検証済みの回路からのみ選択しなければならない。ただし、より長期のガバナンスサイクルにおいては、新しい回路を提案、検証、記録し、将来の Foundation 機構によってプロトタイプ集合へ追加できる。

このトレードオフは意図的である。

Expression freedom $\leftrightarrow$ systemic safety

Phase-2 は、無制約な表現生成よりも、有界な決定論的ガバナンスを優先する。

9. Semantic Transition Model

Semantic Transition とは、ある意味状態または構造 IR から別の意味状態または構造 IR への候補遷移である。

$$\begin{aligned}x_t &\rightarrow x_{t+1} \\ s_t &\rightarrow s_{t+1}\end{aligned}$$

AIKernel において、LLM / SLM はこのような遷移を提案できるが、コミットする権限は持たない。Kernel は Runtime Policy の下で遷移を評価する。

Propose $\rightarrow$ Evaluate $\rightarrow$ Decide $\rightarrow$ Commit

- **Propose:** LLM / SLM が意味状態、IR、回路、または行動を提案する。
- **Evaluate:** AIKernel が受理可能性、距離、能力互換性、ポリシー制約、不変条件を評価する。
- **Decide:** PDP が制御された判断を返す。
- **Commit:** PEP が許可された遷移のみをコミットする。

9.1 Governed Transition

Governed Transition とは、ポリシー評価を通過し、実行可能なシステム状態に影響することを許された意味遷移である。

GovernedTransition(x, c_k) = true
only if admissibility, compatibility, and invariants hold

このモデルは、確率的推論が候補を提案できても、実行権限は直接持たないという Phase-1 の原則を保持する。

10. Semantic Compression and Admissibility

Semantic Compilation は、高エントロピーな Semantic IR を有限の Governed Circuit 候補へ写像する。この写像を **Semantic Compression** または **Semantic Quantization** と呼ぶ。

10.1 Admissibility Function

コンパイラは、入力 IR x が候補回路 c_k へ射影可能かどうかを評価する。

$A(x, c_k) = 1$
iff constraints are preserved
and invariants are preserved
and capabilities are compatible

より具体的には、次のように表せる。

$A(x, c_k) = 1$ if
 $x.C \text{ subseq } c_k.C$
and $x.B \Rightarrow c_k.B$
and $c_k.T$ proves $x.T$
otherwise $A(x, c_k) = 0$

候補回路が一つも受理条件を満たさない場合、コンパイラは Admissibility Error を返し、Fail-Closed に停止する。

forall c_k in P , $A(x, c_k) = 0$
 $\Rightarrow$ AdmissibilityError
 $\Rightarrow$ FailClosed

10.2 複合距離関数

受理可能な候補集合が得られた場合、コンパイラは複合距離関数の下で最も近い Governed Circuit を選択する。

$Q(x) = \operatorname{argmin}_{\{c_k \text{ in } P, A(x, c_k)=1\}} d(x, c_k)$

ここでの距離関数 $d(x, c_k)$ は、単一のユークリッド距離として理解されるべきではない。Semantic IR は、グラフトポロジー、能力要件、ガバナンス制約、境界不変条件という異質な成分から構成される。さらに、各 IR には観測可能な意味状態が関連づけられる。

したがって、本稿では $d(x, c_k)$ を次のような複合距離関数として定義する。より正確には、これは heterogeneous metric aggregation、すなわち異質な距離成分の統合である。各成分は、それぞれに適した構造空間または意味空間で評価され、重み付き和は単一の同質空間上の距離ではなく、運用上のランキングスコアとして用いられる。

$$\begin{aligned} d(x, c_k) &= \lambda_G d_G(G, G_k) \\ &+ \lambda_T d_T(T, T_k) \\ &+ \lambda_C d_C(C, C_k) \\ &+ \lambda_B d_B(B, B_k) \\ &+ \lambda_S d_S(s, s_k) \end{aligned}$$

ここで、 d_G はグラフトポロジー差分、 d_T は能力要件差分、 d_C はガバナンス制約差分、 d_B は境界不変条件差分、 d_S は観測可能な意味状態差分を評価する。

意味状態成分は Mahalanobis-style semantic distance として実装できるが、複合距離関数全体は単一の同質空間上の距離ではない。

$$\begin{aligned} d_S(s, s_k) &= \sqrt{(\mu - \mu_k)^T (\Sigma_k + \epsilon I)^{-1} (\mu - \mu_k)} \end{aligned}$$

ここで、 $\epsilon > 0$ は数値安定化のための微小定数であり、 I は単位行列である。

この定式化により、グラフ構造、能力、制約、不変条件といった離散的な構造差分と、意味状態の不確実性に基づく連続的な距離評価を混同せずに統合できる。

d_S は観測可能性に基づく近似であり、意味の完全性を証明するものではない。これは、観測可能な意味状態と統治可能な回路候補との適合度を評価するための運用上の距離指標である。

10.3 用語上の注意: Quantization は量子力学ではない

本稿における Quantization は、情報理論および信号処理における意味、すなわち連続または高基数の空間を有限の離散代表へ写像することを意味する。物理的な量子力学を意味しない。

11. Deterministic Synthesis

候補回路 c_k が選択されると、Synthesis Layer は構造 IR と選択された回路を、interface contract、DI registration、static SGP graph、tool access policy、replay configuration などのランタイムアーティファクトへ具体化する。

合成関数を次のように表す。

$$S: X \times P \rightarrow R$$

ここで、 R は runtime artifact space である。

11.1 Theorem 1: Deterministic Structural Synthesis

入力 IR、選択された回路、コンパイラバージョン、ポリシー集合、写像規則、テンプレート、およびランタイム環境設定が固定されている場合、合成結果は再現可能である。

$$\begin{aligned} \text{Fixed}(x, c_k, \text{compiler}, \text{policies}, \text{mappings}, \text{templates}) \\ \Rightarrow \text{Var}(S(x, c_k)) = 0 \end{aligned}$$

Proof sketch: 確率的抽出ステップによって記録済み Semantic IR が生成され、Quantization によって記録済み候補回路が選択された後、Synthesis は構造化データからランタイムアーティファクトへの決定論的写像として実装される。合成ステップでは確率的テキスト生成器を呼び出さない。したがって、実装および環境条件が固定されている限り、出力構造は再現可能である。

この定理は、自然言語からの抽出が完全であることを主張しない。主張するのは、IR と回路が固定された後の合成ステップが構造的に決定論的である、という点である。

11.2 制約保存としての Compiler Correctness

古典コンパイラの正当性は、しばしばソースプログラムからターゲットプログラムへの意味保存を扱う。AIKernel では、より慎重な主張が必要である。人間の意図はソースコードのように完全形式化されていない。したがって Phase-2 は、Compiler Correctness を、実行可能な制約、能力、および境界不変条件の保存として定義する。

Preserve(C, B, T) across $S(x, c_k)$

コンパイラは、合成されたランタイムアーティファクトが、コンパイル時に選択された制約 C 、能力 T 、および不変条件 B を強制する場合、その governance profile に対して正しい。

これは、人間の意味がすべて保存されたことの証明ではない。実行に関わるガバナンス制約が保存されたことを示す proof obligation である。

12. Runtime Policy

Runtime Policy は、意味遷移、IR 射影、回路選択が受理可能かを決定する規則集合である。

Runtime Policy には、以下が含まれる。

- capability requirements
- risk thresholds
- token and latency budgets
- isolation requirements
- context provenance requirements
- fail-closed triggers
- replay requirements
- allowed circuit families
- forbidden transition patterns

ポリシーは、遷移を拒否し、明確化を要求し、能力をダウングレードし、スナップショットを要求し、決定論的ソルバーヘルパーティングし、または遷移を Governed Circuit へ受理できる。

Runtime Policy は自由形式の自然言語ではない。バージョン管理され、再現可能で、機械検査可能なポリシーアーティファクトとして表現される必要がある。

13. 既存コンパイラアーキテクチャとの関係

Semantic Compilation はコンパイラアーキテクチャに着想を得ているが、LLVM、Roslyn、または特定のコンパイラ実装と厳密に同型であるとは主張しない。

これはアーキテクチャ上の類推である。

Classical compiler	Semantic compiler
Source code	Concept specification / human intent
Syntax parsing	Semantic extraction
AST / IR	Semantic IR
Optimization	Semantic quantization and policy refinement
Backend	Governed circuit synthesis
Machine code	AIKernel runtime artifact

この類推の目的は、中間表現、許容性エラー、決定論的合成、およびバックエンド選択の役割を明確にすることである。自然言語意味論がプログラミング言語文法と同じ形式的性質を持つ、という主張ではない。

14. Optional Backend Optimization

Prototype Space が拡大すると、最適な Governed Circuit を探索するコストが増大する。グラフポロジのマッチング、部分グラフ互換性、多次元制約最適化は、大規模エンタープライズシステムでは高コストになり得る。

本稿は、Quantization ステップに対して任意のバックエンド最適化機構を許容する。例として、古典的ソルバー、ヒューリスティックなグラフ探索、SMT solver、integer programming、将来的な quantum-inspired optimization があり得る。

これらのバックエンドは最適化機構にすぎない。Semantic IR、Runtime Policy、Admissibility Function、または Deterministic Synthesis Layer を置き換えない。確率的または近似的な最適化器が候補回路を提案する場合でも、最終的な受理と合成は決定論的な検証規則に従う。

15. Applications

15.1 Governed Enterprise Workflow Synthesis

ユーザーが、財務スプレッドシートやコンプライアンス文書に対する複数部署横断監査のような複雑な業務ワークフローを要求した場合、Semantic Compiler は無制約な自律エージェントループを生成しない。代わりに Semantic IR を抽出し、それを Governed Circuit の合成へ写像する。

Retrieval Circuit

- > Quarantine Circuit
- > Cross-Verification Circuit
- > Report Synthesis Circuit

能力、ストレージ境界、リスク閾値、リプレイ要件は、実行開始前に固定される。

15.2 Policy Change による Re-Quantization

企業のコンプライアンス規則、ROM 資産、または VFS capability が変化した場合、既存の Semantic IR アーティファクトの受理可能性も変化し得る。AIKernel は、新しいポリシー集合の下で $A(x, c_k)$ を再評価し、必要に応じて IR を別の Governed Circuit へ再量子化できる。

これは、場当たりのプロンプト書き換えに依存しない、policy-aware runtime evolution の構造モデルを提供する。

16. Theoretical Guarantees and Non-Guarantees

16.1 固定条件下での保証

実装バージョン、ポリシー、写像規則、テンプレート、および記録済み入力固定されている場合、AIKernel は以下を保証できる。

- 決定論的な構造合成
- 受理可能な回路が存在しない場合の Fail-Closed 拒否
- 生成された runtime artifact における宣言済み capability、constraint、invariant の保存
- 回路選択と合成判断の replayability
- intent extraction から runtime artifact までの監査可能な traceability

16.2 保証しないもの

AIKernel は以下を保証しない。

- 人間意図の完全な抽出
- 生成内容の事実正確性
- 自然言語と runtime artifact の完全な意味的等価性
- 意味遷移の大域的な数学的安定性
- 敵対的入力への完全な免疫

- 将来の Phase-3 暗号・来歴証明機構の正しさ

この有界な主張構造は意図的である。Phase-2 は、意味に関する普遍理論ではなく、ガバナンス指向の Semantic Runtime を定義する。

17. Limitations and Future Work

17.1 Prototype Expansion Problem

有限の Prototype Space は安全性を高めるが、表現自由度を下げる。未知のユーザー意図は、保守的な回路へ圧縮され、表現力の損失が生じ得る。今後の作業では、新しい回路プロトタイプを安全に提案、検証、記録、受理する仕組みを定義する必要がある。

17.2 Extraction Uncertainty

自然言語意図から Semantic IR への抽出自体も不確実性を持ち得る。抽出が決定論的に、または十分な信頼度で行えない場合、コンパイラは Clarify を返すか、Fail-Closed しなければならない。

17.3 Proof-Carrying Synthesis

将来的には、合成された runtime artifact に machine-checkable proof や signed attestation を添付できる可能性がある。これは、Digital Deed、provenance、ROM Identity、cryptographic addressability といった Phase-3 Foundation の主題へ接続される。

17.4 Runtime Calibration

ReplayLog データは、距離重み、受理閾値、回路選択規則のキャリブレーションに利用できる。ただし、キャリブレーションはハードポリシー制約や Fail-Closed 境界を置き換えてはならない。

18. Conclusion

Semantic Compilation Architecture は、AIKernel / AIOS Phase-2 の理論である。これは、Phase-1 で定義された静的境界が、意味、観測可能性、および統治された遷移のランタイム理論へどのように展開されるかを説明する。

本稿の中核的貢献は、Prompt Execution から Semantic Compilation への転換である。AIKernel は自然言語を直接実行可能なものとして扱わない。高エントロピーな人間意図を Semantic IR へ変換し、有限の Governed Circuit からなる Prototype Space との適合性を評価し、明示的な Runtime Policy の下で決定論的な runtime artifact へ合成する。

このモデルは、確率的知能そのものを決定論化するものではない。確率的な提案を決定論的な統治境界の内側に置くものである。また、意味の完全性を証明するものではなく、意味を含む遷移を観測・統治・再現・監査するための運用構造を定義する。

この意味で、Phase-2 は Phase-1 と Phase-3 を接続する。AIKernel の静的 OS 境界を、将来的な provenance、identity、proof、cryptographic trust へ接続可能な Semantic Runtime Theory へ変換するのである。

References

1. Shannon, Claude E. “A Mathematical Theory of Communication.” Bell System Technical Journal, vol. 27, no. 3, 1948, pp. 379-423. DOI: 10.1002/j.1538-7305.1948.tb01338.x.
2. Mahalanobis, Prasanta Chandra. “On the Generalized Distance in Statistics.” Proceedings of the National Institute of Sciences of India, vol. 2, no. 1, 1936, pp. 49-55. Available at: https://insa.nic.in/writereaddata/UploadedFiles/PINSA/Vol02_1936_1_Art05.pdf.

3. Lattner, Chris, and Vikram Adve. "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation." International Symposium on Code Generation and Optimization (CGO 2004), 2004, pp. 75-88. DOI: 10.1109/CGO.2004.1281665.
4. Sikka, Varin, and Vishal Sikka. "Hallucination Stations: On Some Basic Limitations of Transformer-Based Language Models." arXiv:2507.07505, 2025. DOI: 10.48550/arXiv.2507.07505.
5. National Institute of Standards and Technology. Guide to Attribute Based Access Control (ABAC) Definition and Considerations. NIST Special Publication 800-162, 2014. DOI: 10.6028/NIST.SP.800-162.
6. OASIS. eXtensible Access Control Markup Language (XACML) Version 3.0. OASIS Standard, 22 January 2013. Available at: <http://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html>.
7. Sogawa, Takuya. "Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model." Zenodo, 2026. DOI: 10.5281/zenodo.20290614.
8. Sogawa, Takuya. "AIKernel Phase-1 Paper 01: ROM Format and Knowledge Snapshot Model." Zenodo, 2026. DOI: 10.5281/zenodo.20306539.
9. Sogawa, Takuya. "AIKernel Phase-1 Paper 02: VFS Architecture and Semantic Storage Model." Zenodo, 2026. DOI: 10.5281/zenodo.20307891.
10. Sogawa, Takuya. "AIKernel Phase-1 Paper 03: Pre-Inference Admissibility Governance." Zenodo, 2026. DOI: 10.5281/zenodo.20308639.
11. Sogawa, Takuya. "AIKernel Phase-1 Paper 04: Trajectory Governance Model." Zenodo, 2026. DOI: 10.5281/zenodo.20309510.